

CRC Cyber Lab

[CMMC Level 1 Identification & Authentication \(IA\) Labs — Student Completion Guide](#)

[Table of Contents](#)

[Before You Begin](#)

[How to Connect to the Lab Environment](#)

[How to Open Your Tools](#)

[Understanding Your Pod](#)

[Key Paths and Locations](#)

[Module 1: User Identification](#)

[Module 2: Non-Person Entity Identification](#)

[Module 3: User Authentication Management](#)

[Module 4: Defaults & Process Authentication](#)

[Quick Reference: Tools and Commands](#)

[Tips and Common Mistakes](#)

[Lab Completion Checklist](#)

CMMC Level 1 Identification & Authentication (IA) Labs — Student Completion Guide

This guide provides step-by-step instructions for completing all 12 Identification & Authentication labs. Each lab presents a real-world identity management problem that you must find and fix using tools on the domain controller: Active Directory Users and Computers (ADUC), PowerShell, Task Scheduler, and file-based evidence artifacts.

Table of Contents

1. [Before You Begin](#)
 2. [How to Connect to the Lab Environment](#)
 3. [How to Open Your Tools](#)
 4. [Understanding Your Pod](#)
 5. [Key Paths and Locations](#)
 6. [Module 1: User Identification \(Labs M1-L1 – M1-L3\)](#)
 7. [Module 2: Non-Person Entity Identification \(Labs M2-L1 – M2-L3\)](#)
 8. [Module 3: User Authentication Management \(Labs M3-L1 – M3-L3\)](#)
 9. [Module 4: Defaults & Process Authentication \(Labs M4-L1 – M4-L3\)](#)
 10. [Quick Reference: Tools and Commands](#)
 11. [Tips and Common Mistakes](#)
 12. [Lab Completion Checklist](#)
-

Before You Begin

What You Need

- Your **Pod number** (your instructor will assign this, e.g., Pod01, Pod05, Pod12)
- Your **Guacamole login credentials** (your instructor will provide your username and password)
- A password you choose for the accounts you create: **at least 12 characters** with an upper-case letter, a lower-case letter, a number and a symbol (for example LabUser!2026#ia). Your own sign-in password is shorter than this and will be rejected
- A computer with a web browser (Chrome, Firefox, or Edge) — no special software needed

What You Will Be Doing

You will work on a **Windows Server domain controller** to identify and remediate identity and authentication problems. Unlike the Access Control labs (which focus on group membership), the IA labs use a wider range of tools:

- **Active Directory Users and Computers (ADUC)** — managing user accounts and properties
- **PowerShell** — exporting reports, changing password policy, inspecting scheduled tasks
- **Task Scheduler** — viewing and reconfiguring the account a scheduled task runs as
- **File Explorer / Notepad** — creating evidence files, CSV inventories, and remediation summaries

CMMC Context

These labs align with **CMMC Level 1 Identification & Authentication (IA)** requirements:

- **IA.L1-3.5.1** — Identify system users, processes acting on behalf of users, and devices
 - **IA.L1-3.5.2** — Authenticate (or verify) the identities of those users, processes, or devices as a prerequisite to allowing access
-

How to Connect to the Lab Environment

You will connect to the lab through **Apache Guacamole** — a web-based remote desktop gateway. There is nothing to install; everything runs in your web browser.

Step 1: Open the Guacamole Gateway

1. Open your web browser (Chrome, Firefox, or Edge)
2. Go to: <https://guac.01.digitalrcc.com/#/>
3. You will see a login screen

Step 2: Log In with Your Student Credentials

1. Enter your **Username** — this matches your pod number:
 - Pod 01 → student01
 - Pod 05 → student05
 - Pod 12 → student12
 - (and so on — the number matches your assigned pod)
2. Enter your **Password** (provided by your instructor)
3. Click **Login**

Step 3: Connect to the Domain Controller

Connection Name	What It Is
PODXX-DC	Domain Controller — use this for all IA labs

This is the only connection you get: every IA lab is done on this desktop.

1. Click on **PODXX-DC** (where XX is your pod number, e.g., **POD03-DC**)
2. The remote desktop session opens directly in your browser — no extra login is needed
3. Wait a few seconds for the Windows Server desktop to appear

Tip: Press **Ctrl+Alt+Shift** to open the Guacamole side menu (to return Home or switch connections).

Step 4: Check Your Lab Progress

Option 1: Click the "Check Your Progress — Pod XX" banner at the top of the Guacamole interface.

Option 2: Go directly to <https://training.digitalrcc.com/pod/XX> (replace XX with your pod number).

How to Open Your Tools

Active Directory Users and Computers (ADUC)

1. On the server desktop, double-click the **Active Directory Users and Computers** shortcut

If the shortcut is missing: press **Windows + R** and paste this line exactly, then press Enter:

```
cmd /c "set __COMPAT_LAYER=RunAsInvoker&& start "" mmc.exe dsa.msc"
```

Do not open ADUC any other way. Plain `dsa.msc`, **Server Manager → Tools** and a blank MMC console all make Windows try to start it elevated and show a **User Account Control** prompt for administrator credentials. Student accounts are deliberately not administrators of the shared domain controller, so that prompt always fails with "Logon failure: the user has not been granted the requested logon type at this computer" — nothing is broken. Click **No** and use the shortcut (or the command above), which runs ADUC with your own delegated pod permissions.

PowerShell

1. Right-click the **Start** button → **Windows PowerShell**
2. Alternative: **Windows + R** → `powershell` → Enter

Run PowerShell normally. Do **not** choose **Windows PowerShell (Admin)** or **Run as administrator** — every command in this guide works with your own delegated permissions, and elevating triggers the same UAC prompt described above.

Task Scheduler (needed for M2-L1)

1. **Windows + R** → `taskschd.msc` → Enter

File Explorer

- **Windows + E**, or the folder icon on the taskbar

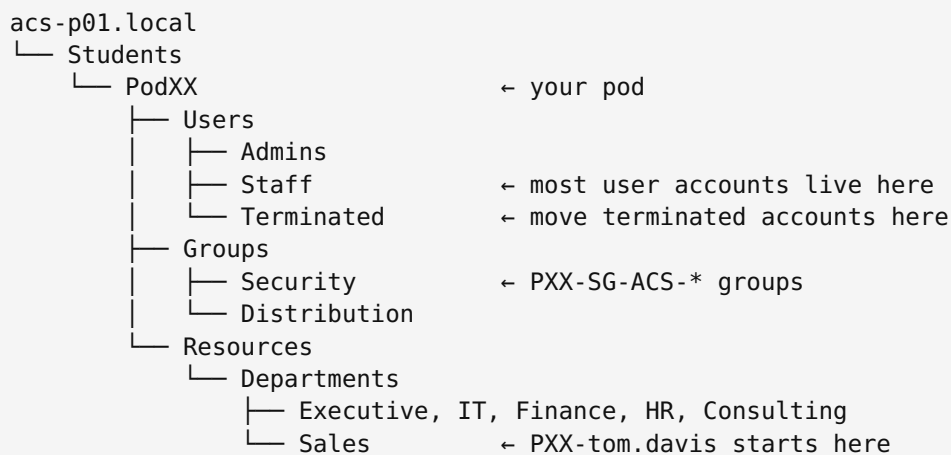
Understanding Your Pod

All pods share one domain (`acs-p01.local`), so **every object in your pod is prefixed with your pod prefix** to keep your work separate. If you are **Pod 03**, your prefix is **P03**:

Thing	Example for Pod 03
Shared reception account	<code>P03-frontdesk</code>
Terminated employee	<code>P03-tom.davis</code>
Generic accounts	<code>P03-admin</code> , <code>P03-user1</code> , <code>P03-test</code>
Backup task owner	<code>P03-s.jenkins</code>
Service accounts	<code>P03-svc_backup</code> , <code>P03-svc_web</code> , <code>P03-svc_print</code>
Security group	<code>P03-SG-ACS-Sales</code>
Scheduled task	<code>Pod03 ACS Nightly Backup</code>
Evidence root	<code>C:\CyberLab\Pod03\</code>

Critical: when you create an account, you must include your prefix (`PXX-k.omalley`, **not** `k.omalley`). The automated checker looks for the prefixed name.

Your OU Structure in ADUC



Tip: in ADUC use **Edit → Find** and search for your prefix (e.g. P03-) to list all of your pod's objects quickly.

Key Paths and Locations

All IA files live under `C:\CyberLab\PodXX\` on the domain controller (replace XX with your pod number):

Path	Purpose
<code>C:\CyberLab\PodXX\</code>	Evidence root — write <code>M1-L1.txt</code> , <code>M3-L1.txt</code> , <code>M3-L3.txt</code> , <code>M4-L1.txt</code> , <code>M4-L3.txt</code> here
<code>C:\CyberLab\PodXX\IA-Artifacts\</code>	IA artifacts (CSV inventories, reports, device list, hardening standard)
<code>C:\CyberLab\PodXX\IA-Artifacts\Vault\</code>	Vault entries file (Module 4)
<code>C:\CyberLab\PodXX\LabArtifacts\</code>	Support files placed by the seed process (e.g. <code>rogue_mac.txt</code>)
<code>C:\CyberLab\PodXX\LabArtifacts\Scripts\</code>	Script to review and fix (<code>db_connect.py</code>)
<code>C:\CyberLab\PodXX\LabArtifacts\Scans\</code>	Scan report to analyze (<code>openvas_scan_report.txt</code>)
<code>C:\CyberLab\PodXX_LAB_READY_IA-*.txt</code>	Markers confirming each lab was seeded

Important: the automated verification checks these exact paths and file names, including your pod folder. Saving to `C:\CyberLab\IA-Artifacts\` (without PodXX) will fail.

Module 1: User Identification

Module 1 is about making sure every person has a unique, individual, accounted-for identity — and that shared, zombie, and generic accounts are remediated.

Lab M1-L1 — Shared Reception Account

Difficulty: Beginner | **Time:** 20 minutes | **Type:** Hands-on (ADUC)

Scenario

The front desk uses one shared account, `PXX-frontdesk`, that multiple reception staff log into. Shared accounts break individual accountability. Two people use it: Karen O'Malley and a temporary agency worker.

Your Task

1. Disable the shared account
2. Create an individual account for each person
3. Write an evidence file

Steps

1. Disable the shared account:

- In ADUC, go to **acs-p01.local** → **Students** → **PodXX** → **Users** → **Staff**
- Right-click **PXX-frontdesk** → **Disable Account** → OK

2. Create Karen O'Malley's account:

- Right-click the **Staff** OU → **New** → **User**
- **First name:** Karen, **Last name:** OMalley
- **User logon name:** `PXX-k.omalley` — the prefix is required
- Click **Next**, enter a password that meets the domain policy (12+ characters with upper case, lower case, a number and a symbol, e.g. `LabUser!2026#ia`), uncheck "User must change password at next logon", click **Next** → **Finish**

3. Create the temporary worker's account the same way, with logon name `PXX-temp.agency01`

4. Write the evidence file:

- Open **Notepad** and write a short summary, for example:

```
IA M1-L1 Remediation
Date: 2026-06-10   Performed by: <your name>
- Disabled shared account PXX-frontdesk
- Created individual account PXX-k.omalley (Karen O'Malley)
- Created individual account PXX-temp.agency01 (Temporary Agency Worker)
```

- Save as `C:\CyberLab\PodXX\M1-L1.txt`

PowerShell alternative:

```
$p = "PXX" # your prefix, e.g. P03
Disable-ADAccount -Identity "$p-frontdesk"
Get-ADUser -Identity "$p-frontdesk" | Select-Object SamAccountName, Enabled
```

Completion Criteria

- ☐ PXX-frontdesk exists and is **disabled**
- ☐ PXX-k.omalley exists
- ☐ PXX-temp.agency01 exists
- ☐ `C:\CyberLab\PodXX\M1-L1.txt` exists

Why This Matters

IA.L1-3.5.1 requires every system user to be individually identified. With a shared account there is no way to tell who did what.

Lab M1-L2 — Zombie Account (Terminated User Still Enabled)

Difficulty: Beginner | **Time:** 20 minutes | **Type:** Hands-on (ADUC)

Scenario

Tom Davis left the company, but PXX-tom.davis is still enabled, still in the Sales OU, and still a member of the Sales security group. That is a "zombie account".

Your Task

1. Disable the account
2. Remove its group memberships
3. Move it to the **Terminated** OU

Steps

1. **Find the account:** ADUC → **Students** → **PodXX** → **Resources** → **Departments** → **Sales** (or **Edit** → **Find** for PXX-tom.davis)
2. **Disable it:** double-click the account → **Account** tab → check "**Account is disabled**" → **Apply**
3. **Strip group memberships:** **Member Of** tab → select PXX-SG-ACS-Sales → **Remove** → **Yes** → remove any other groups (Domain Users cannot be removed) → **Apply** → **OK**
4. **Move it:** right-click PXX-tom.davis → **Move...** → **Students** → **PodXX** → **Users** → **Terminated** → OK

PowerShell alternative:

```
$p = "PXX"
Disable-ADAccount -Identity "$p-tom.davis"
Get-ADUser "$p-tom.davis" -Properties MemberOf | Select-Object -ExpandProperty MemberOf
    |
    ForEach-Object { Remove-ADGroupMember -Identity $_ -Members "$p-tom.davis" -Confirm:
        $false }
$parent = (Get-ADUser "$p-tom.davis").DistinguishedName.Split(',', 2)[1]
$ou = (Get-ADOrganizationalUnit -Filter "Name -eq 'Terminated'" `
    -SearchBase $parent).DistinguishedName
Move-ADObject -Identity (Get-ADUser "$p-tom.davis").DistinguishedName -TargetPath $ou
```

(If the last two lines are awkward, just move the account in ADUC.)

Completion Criteria

- ☐ PXX-tom.davis is **disabled**
- ☐ No group memberships remain except Domain Users
- ☐ The account's location is inside the **Terminated** OU

Why This Matters

Former employees' credentials show up in password dumps. A disabled, stripped, relocated account removes that entry point and shows a controlled termination process.

Lab M1-L3 — Generic Accounts Present

Difficulty: Beginner | **Time:** 25 minutes | **Type:** Hands-on (ADUC) + CSV

Scenario

Three generic accounts exist in your pod: PXX-admin, PXX-user1, and PXX-test. None can be traced to a person, and there is no inventory of authorized users.

Your Task

1. Disable the three generic accounts
2. Create an authorized user inventory CSV

Steps

1. **Disable each generic account:** ADUC → **Students** → **PodXX** → **Users** → right-click each of PXX-admin, PXX-user1, PXX-test → **Disable Account**

PowerShell alternative:

```
$p = "PXX"
"$p-admin", "$p-user1", "$p-test" | ForEach-Object { Disable-ADAccount -Identity $_ }
```

2. **Create the inventory.** Open Notepad and enter something like:

```
Username,FullName,Department,Status,Justification
PXX-k.omalley,Karen OMalley,Reception,Active,Individual reception account
PXX-temp.agency01,Temp Agency Worker,Reception,Active,Temporary agency staff
PXX-tom.davis,Tom Davis,Sales,Disabled,Terminated employee
PXX-admin,Admin Account,N/A,Disabled,Generic account - no individual owner
PXX-user1,User1 Account,N/A,Disabled,Generic account - no individual owner
PXX-test,Test Account,N/A,Disabled,Generic account - no individual owner
```

3. **Save as** C:\CyberLab\PodXX\IA-Artifacts\Authorized_User_Inventory.csv
 - In the Notepad **Save as type** box choose **All Files (*.*)** so it does not become .csv.txt

Completion Criteria

☐

PXX-admin, PXX-user1, PXX-test are all disabled (or deleted)

☐

Authorized_User_Inventory.csv exists in your pod's IA-Artifacts folder and is not empty

Why This Matters

If "admin" does something malicious, nobody can say who that was. An authorized user inventory proves the organization knows and approves every account it has.

Module 2: Non-Person Entity Identification

Module 2 covers the identities that are not people — service accounts, automated processes, and devices. CMMC requires them to be identified and managed just like human users.

Lab M2-L1 — Scheduled Task Running as a Human Account

Difficulty: Intermediate | **Time:** 10 minutes | **Type:** Hands-on (ADUC + Task Scheduler)

Where the task lives: PodXX ACS Nightly Backup is a task on **your own pod server**, not on the domain controller, and you are a local administrator there — so you make the change yourself and both steps below are graded. Open Task Scheduler from an elevated session (right-click → **Run as administrator**) so Windows lets you save the task credential.

Scenario

The nightly backup task **PodXX ACS Nightly Backup** runs under a real employee's account, PXX-s.jenkins. Automated jobs should run as dedicated service accounts: when Steve leaves, the backup breaks, and his credentials are needlessly exposed.

Your Task

1. Create a dedicated service account PXX-svc_backup
2. Reconfigure the task to run as it

Steps

1. Create the service account:

- ADUC → **Students** → **PodXX** → **Users** → right-click → **New** → **User**
- **Full name / logon name:** PXX-svc_backup
- Click **Next**, enter a password that meets the domain policy (12+ characters with upper case, lower case, a number and a symbol), check "**Password never expires**", uncheck "User must change password at next logon" → **Next** → **Finish**
- Double-click the account and add a **Description:** Service account for nightly backup automation

2. Point the task at the service account:

- Open **Task Scheduler** (**Windows + R** → taskschd.msc)
- Click **Task Scheduler Library** and find **PodXX ACS Nightly Backup**
- Right-click → **Properties** → **General** tab
- The "When running the task, use the following user account" field shows ACS-P01\PodXX-s.jenkins

- Click **Change User or Group...**, type `PXX-svc_backup`, click **Check Names** → **OK**
- Click **OK** and enter the service account password when prompted

3. Confirm the change:

```
(Get-ScheduledTask -TaskName "PodXX ACS Nightly Backup").Principal.UserId
```

It should now return `ACS-P01\PXX-svc_backup`.

If Windows warns "Batch logon privilege needs to be enabled": the task is saved and graded, but it cannot actually start until the service account holds that right. Grant it on your own server in **secpol.msc** → **Local Policies** → **User Rights Assignment** → **Log on as a batch job**, add `PXX-svc_backup`, and reopen the task to confirm it saves without the warning.

Completion Criteria

☐

`PXX-svc_backup` exists in Active Directory

☐

`PodXX ACS Nightly Backup` on your pod server runs as `PXX-svc_backup`, not `PXX-s.jenkins`

Why This Matters

IA.L1-3.5.1 covers "processes acting on behalf of users". A dedicated service account gives the automated process its own identity and its own accountability.

Lab M2-L2 — Rogue Device Artifact

Difficulty: Beginner | **Time:** 20 minutes | **Type:** CSV analysis

Scenario

A device scan detected MAC address `AA-BB-CC-11-22-33`, which is not in the authorized device list. You must record it as unauthorized and open a configuration record for investigation.

Your Task

1. Review the authorized device list and the rogue MAC file
2. Add the rogue device to the list, marked `UNAUTHORIZED`
3. Create a device configuration record CSV

Steps

1. Review the evidence:

- Open `C:\CyberLab\PodXX\IA-Artifacts\Authorized_Device_List.csv` — three authorized devices (DC, workstation, printer)
- Open `C:\CyberLab\PodXX\LabArtifacts\rogue_mac.txt` — the detected MAC AA-BB-CC-11-22-33

2. Add the rogue device to the bottom of `Authorized_Device_List.csv` (open it in Notepad), keeping the existing columns:

```
UNKNOWN-DEVICE,AA-BB-CC-11-22-33,Unknown,UNAUTHORIZED,Unknown,2026-06-10
```

Save the file (do not delete the existing rows).

3. Create the configuration record. Open Notepad and enter:

```
MACAddress,DeviceName,Status,Location,Notes  
AA-BB-CC-11-22-33,Unknown Device,UNAUTHORIZED,Network scan,Rogue device detected -  
requires investigation
```

Save as `C:\CyberLab\PodXX\IA-Artifacts\Device_Config_Record.csv` (**All Files** type).

Heads-up: Lab **M4-L2** adds a second finding to this same `Device_Config_Record.csv`. When you get there, **append** to the file — do not overwrite it, or M2-L2 will stop passing.

Completion Criteria

☐

`Authorized_Device_List.csv` contains AA-BB-CC-11-22-33 **and** the word UNAUTHORIZED

☐

`Device_Config_Record.csv` exists and is not empty

Why This Matters

IA.L1-3.5.1 requires devices to be identified, not just users. An undocumented device on the network is an unmanaged entry point.

Lab M2-L3 — Service Account Matrix Required

Difficulty: Beginner | **Time:** 25 minutes | **Type:** Hands-on (ADUC) + CSV

Scenario

Three service accounts exist — PXX-svc_backup, PXX-svc_web, PXX-svc_print — but their **Description** fields are empty, and there is no matrix documenting the non-person accounts.

Your Task

1. Populate the Description field on all three accounts
2. Create a service account matrix CSV

Steps

1. **Set the descriptions** in ADUC (**Students** → **PodXX** → **Users**) — double-click each account, fill **Description**, click OK:

- PXX-svc_backup → Automated nightly backup service
- PXX-svc_web → IIS web application pool identity
- PXX-svc_print → Print spooler service account

PowerShell alternative:

```
$p = "PXX"
Set-ADUser "$p-svc_backup" -Description "Automated nightly backup service"
Set-ADUser "$p-svc_web" -Description "IIS web application pool identity"
Set-ADUser "$p-svc_print" -Description "Print spooler service account"
```

2. **Create the matrix.** In Notepad:

```
AccountName,Description,Owner,Purpose>PasswordRotation,LastReview
PXX-svc_backup,Automated nightly backup service,IT Operations,Runs the nightly
backup scheduled task,90 days,2026-04-01
PXX-svc_web,IIS web application pool identity,Web Team,Hosts internal web
applications,90 days,2026-04-01
PXX-svc_print,Print spooler service account,IT Operations,Manages network print
services,90 days,2026-04-01
```

Save as C:\CyberLab\PodXX\IA-Artifacts\Service_Account_Matrix.csv.

Completion Criteria



All three service accounts exist with a non-empty Description



Service_Account_Matrix.csv exists and is not empty

Note: If PXX-svc_web or PXX-svc_print is missing, create it the same way you created PXX-svc_backup in M2-L1.

Why This Matters

An undocumented service account is an identity nobody owns. The matrix records what it does, who owns it, and when it was last reviewed — the basis of lifecycle management for non-person identities.

Module 3: User Authentication Management

Module 3 is about how users prove who they are: password policy, documented review, and credential handling.

Lab M3-L1 — Password Policy Report Missing

Difficulty: Beginner | **Time:** 15 minutes | **Type:** PowerShell + evidence

Scenario

A domain password policy exists, but there is no exported report and no record that anyone reviewed it. Auditors need written evidence.

Your Task

1. Export the current password policy to an HTML report
2. Write an evidence file describing your review

Steps

1. **Export the policy** (PowerShell, replace PodXX):

```
Get-ADDefaultDomainPasswordPolicy |  
    ConvertTo-Html -Title "Password Policy Report" |  
    Out-File "C:\CyberLab\PodXX\IA-Artifacts\PasswordPolicy_Report.html"
```

2. **Open and read it:**

```
Start-Process "C:\CyberLab\PodXX\IA-Artifacts\PasswordPolicy_Report.html"
```

Note MinPasswordLength, ComplexityEnabled, LockoutThreshold, PasswordHistoryCount, MaxPasswordAge.

3. Write the evidence file in Notepad and save as C:\CyberLab\PodXX\M3-L1.txt:

```
IA M3-L1 Password Policy Review
Date: 2026-06-10   Reviewed by: <your name>
```

```
Current policy settings:
```

- Minimum password length: <value>
- Password complexity: <Enabled/Disabled>
- Lockout threshold: <value>
- Password history: <value>
- Maximum password age: <value> days

```
Assessment: policy exported and reviewed as required by CMMC IA.L1-3.5.2.
```

Completion Criteria

☐

PasswordPolicy_Report.html exists in your pod's IA-Artifacts folder and is not empty

☐

C:\CyberLab\PodXX\M3-L1.txt exists

Why This Matters

Having a policy is not enough — you must be able to show it exists and is reviewed. Missing documentation is one of the most common audit findings.

Lab M3-L2 — Weak Password Policy

Difficulty: Intermediate | **Time:** 15 minutes | **Type:** Hands-on (Local Security Policy)

Where the policy lives: you harden the account policy of **your own pod server**, which governs the local accounts on that server. The domain password policy is shared by all 20 pods and stays with the instructors, so nothing you do here affects anyone else's pod. Work from an elevated session.

Scenario

Your server's local account policy is dangerously weak: minimum length 6, complexity disabled, and no account lockout. An attacker who reaches a local account can brute-force it in seconds.

Required Settings

Setting	Seeded (FAIL)	Required (PASS)
Minimum password length	6	12 or more
Password complexity	Disabled	Enabled
Account lockout threshold	0 (no lockout)	1-10 invalid attempts

Steps

1. Read the policy you start with:

```
net accounts
```

2. Harden it — GUI (secpol.msc):

- Press **Windows + R**, type `secpol.msc`, press Enter (accept the UAC prompt)
- **Account Policies → Password Policy**: set Minimum password length to `12`, and Password must meet complexity requirements to **Enabled**
- **Account Policies → Account Lockout Policy**: set Account lockout threshold to `10` invalid logon attempts (Windows will offer lockout duration and reset counter defaults — accept them)

PowerShell alternative (run elevated; complexity has no `net accounts` switch, so it is set through the security database):

```
net accounts /minpwlen:12 /lockoutthreshold:10 /uniquepw:24

$inf = "$env:TEMP\pwpolicy.inf"
@'
[Unicode]
Unicode=yes
[Version]
signature="$CHICAGO$"
Revision=1
[System Access]
PasswordComplexity = 1
'@ | Set-Content -LiteralPath $inf -Encoding Unicode
secedit /configure /db "$env:TEMP\pwpolicy.sdb" /cfg $inf /areas SECURITYPOLICY
```

- ### 3. Confirm your work
- `net accounts` should now report minimum length 12 and a lockout threshold of 10, and `secedit /export /cfg "$env:TEMP\now.cfg"` should show `PasswordComplexity = 1`.

Completion Criteria



Minimum password length on your pod server is 12 or more



Password complexity is enabled



Account lockout threshold is between 1 and 10 invalid attempts

Why This Matters

A 6-character password with no complexity and no lockout can be brute-forced in seconds.

IA.L1-3.5.2 requires authentication that actually resists common attacks — on every system in scope, not just the domain.

Lab M3-L3 — User Must Change Password at Next Logon

Difficulty: Beginner | **Time:** 15 minutes | **Type:** Hands-on (ADUC) + evidence

Scenario

David Chen (PXX-d.chen) had his password reset by the helpdesk, but the "must change password at next logon" flag was never set — so he can keep using the temporary password forever, and whoever set it still knows it.

Your Task

1. Set the must-change-password flag on PXX-d.chen
2. Write an incident evidence file

Steps

1. **In ADUC: Students → PodXX → Users → Staff** → right-click PXX-d.chen → **Reset Password...** → enter a new temporary password → check **"User must change password at next logon"** → OK

PowerShell alternative:

```
$p = "PXX"
Set-ADUser -Identity "$p-d.chen" -ChangePasswordAtLogon $true
Get-ADUser "$p-d.chen" -Properties pwdLastSet | Select-Object SamAccountName,
    pwdLastSet # expect 0
```

2. **Verify** in the account's **Account** tab that the flag is set (or that pwdLastSet is 0)
3. **Write the evidence file** and save as C:\CyberLab\PodXX\M3-L3.txt:

```
IA M3-L3 Password Reset Incident
Date: 2026-06-10   Remediated by: <your name>
User: PXX-d.chen (David Chen)
Issue: password was reset without "must change at next logon"
Action: reset the password and enabled the must-change-at-next-logon flag
Result: the user must set their own password at next sign-in
```

Completion Criteria

☐

PXX-d.chen has the must-change-password flag set (`pwdLastSet = 0`)

☐

`C:\CyberLab\PodXX\M3-L3.txt` exists

Why This Matters

A temporary password that is never changed is a shared secret between the user and whoever issued it — which defeats individual authentication.

Module 4: Defaults & Process Authentication

Module 4 covers default credentials, hardcoded passwords, and undocumented authentication configuration — among the most commonly exploited weaknesses in real attacks.

Lab M4-L1 — Default Credentials Not Addressed in the Hardening Standard

Difficulty: Beginner | **Time:** 20 minutes | **Type:** Document editing + evidence

Scenario

Your organization has a hardening standard (`Hardening_Standard.txt`), but it never requires default passwords to be changed on new equipment.

Your Task

1. Review the standard
2. Add a default-credentials clause
3. Write an evidence file

Steps

1. **Read** `C:\CyberLab\PodXX\IA-Artifacts\Hardening_Standard.txt` — it covers OS hardening, network security, accounts, logging, and physical security, and ends by admitting it is incomplete

2. **Append a section** in Notepad and save the file:

6. DEFAULT CREDENTIALS

- All default passwords on newly deployed hardware and software must be changed before the system is placed into production, including: network equipment (routers, switches, firewalls), server OS default accounts, application default admin accounts, SNMP community strings, and database default credentials.
- Responsibility: IT Operations must verify default credentials are changed as part of the deployment checklist.
- Reference: CMMC IA.L1-3.5.2 - authenticate identities before granting access

The checker looks for wording that ties **default** to **password/credential** — keep those words in the text.

3. **Write the evidence file** and save as `C:\CyberLab\PodXX\M4-L1.txt`:

```
IA M4-L1 Hardening Standard Update
Date: 2026-06-10   Updated by: <your name>
Action: added a default credentials section to Hardening_Standard.txt
Location: C:\CyberLab\PodXX\IA-Artifacts\Hardening_Standard.txt
The standard now requires all default passwords to be changed before
production deployment, per CMMC IA.L1-3.5.2.
```

Completion Criteria

☐

`Hardening_Standard.txt` contains a default password/credential requirement

☐

`C:\CyberLab\PodXX\M4-L1.txt` exists

Why This Matters

Default credentials are the easiest way in. The Mirai botnet took over hundreds of thousands of devices using nothing else.

Lab M4-L2 — SNMP Community String Set to "public"

Difficulty: Beginner | **Time:** 20 minutes | **Type:** Scan report analysis + CSV

Scenario

A vulnerability scan flagged a network device using the default SNMP community string `public` — anyone on the network can read its configuration.

Your Task

1. Review the scan report

2. Document the SNMP finding in the device configuration record
3. Add an SNMP clause to the hardening standard

Steps

1. **Read the scan report:** `C:\CyberLab\PodXX\LabArtifacts\Scans\openvas_scan_report.txt`
 - The [HIGH] finding is SNMP community string `public` on **10.50.1.30 (PRINT-P01)**, port 161/udp
2. **Append the finding to** `C:\CyberLab\PodXX\IA-Artifacts\Device_Config_Record.csv`.

Do not overwrite this file — it still has to contain your M2-L2 rogue-device row (AA-BB-CC-11-22-33 / UNAUTHORIZED) or M2-L2 will fail. Open it in Notepad and add a line at the end, for example:

```
10.50.1.30,PRINT-P01,SNMP community string is 'public' (HIGH) - change to a complex value and restrict SNMP,OPEN,Network scan,From openvas_scan_report.txt
```

Keeping both findings in one file is what the checker expects: the file must mention `SNMP (M4-L2)` **and** the rogue MAC marked `UNAUTHORIZED` must still be in `Authorized_Device_List.csv (M2-L2)`.

3. **Add an SNMP clause** to `Hardening_Standard.txt` (good practice, and it reinforces M4-L1):

```
7. SNMP CONFIGURATION
- Default SNMP community strings (public, private) must be changed on all network devices. Use SNMPv3 with authentication and encryption where supported. SNMP v1/v2c with default strings is a critical finding.
```

Completion Criteria



`Device_Config_Record.csv` mentions `SNMP`



The M2-L2 rogue-device evidence is still intact

Why This Matters

SNMP with `public` hands out interface details, routing tables, and sometimes credentials to anyone who asks. Documenting the finding is the first step of correcting it.

Lab M4-L3 — Script Contains a Hardcoded Password

Difficulty: Intermediate | **Time:** 25 minutes | **Type:** Hands-on (file editing) + evidence

Scenario

`db_connect.py` connects to the inventory database using a hardcoded password (`password123`). Anything that touches that file — a repo, a backup, a file share — leaks the credential.

Your Task

1. Replace the hardcoded password with a vault reference
2. Create a vault entries file
3. Write a remediation summary

Steps

1. **Open** `C:\CyberLab\PodXX\LabArtifacts\Scripts\db_connect.py` in Notepad and find:

```
DB_PASSWORD = "password123"
```

Change it to a vault reference and save:

```
DB_PASSWORD = "VAULT_REF:db_production_password"
```

The literal `password123` must no longer appear anywhere in the file, and `VAULT_REF` must appear.

2. **Create the vault entries file.** In Notepad:

```
VaultKey,Description,Owner,RotationSchedule,LastRotated  
db_production_password,Production database connection credential,DBA Team,90 days,  
2026-04-01
```

Save as `C:\CyberLab\PodXX\IA-Artifacts\Vault\Vault_Entries.txt`

3. **Write the remediation summary** and save as `C:\CyberLab\PodXX\M4-L3.txt`:

IA M4-L3 Hardcoded Password Remediation
 Date: 2026-06-10 Remediated by: <your name>
 Finding: db_connect.py contained the hardcoded password "password123"
 Location: C:\CyberLab\PodXX\LabArtifacts\Scripts\db_connect.py

Remediation:

1. Replaced the hardcoded password with VAULT_REF:db_production_password
2. Documented the credential in IA-Artifacts\Vault\Vault_Entries.txt
3. The password is now retrieved from the vault at runtime, not stored in code

Impact: eliminated credential exposure through script files.

Completion Criteria

- ☐ db_connect.py no longer contains password123
- ☐ db_connect.py contains VAULT_REF
- ☐ Vault_Entries.txt exists in IA-Artifacts\Vault\ and is not empty
- ☐ C:\CyberLab\PodXX\M4-L3.txt exists

Why This Matters

Hardcoded credentials are CWE-798, a top software weakness. They end up in version control, backups, and logs. A vault reference keeps the secret out of code and makes rotation possible.

Quick Reference: Tools and Commands

PowerShell Commands Used in the IA Labs

Command	Purpose
Get-ADUser -Filter "SamAccountName -eq 'PXX-name'"	Find a specific AD user
Disable-ADAccount -Identity "PXX-name"	Disable a user account
New-ADUser -SamAccountName "PXX-name" -Name "PXX-name" ...	Create a new user
Set-ADUser -Identity "PXX-name" -Description "text"	Set a user description
Set-ADUser -Identity "PXX-name" -ChangePasswordAtLogon \$true	Force a password change
Get-ADDefaultDomainPasswordPolicy	View the current password policy
Set-ADDefaultDomainPasswordPolicy	Change the password policy
Get-ScheduledTask -TaskName "PodXX ACS Nightly Backup"	View the backup scheduled task

Command	Purpose
gpupdate /force	Force a Group Policy refresh

File Operations

Task	How To
Create a text file	Notepad → File → Save As → choose the path and name
Create a CSV file	Notepad → type comma-separated data → Save As → change type to All Files → name it .csv
Open a path quickly	Paste the path into the File Explorer address bar
Check a file exists	Test-Path "C:\CyberLab\PodXX\M1-L1.txt"

Keyboard Shortcuts

Shortcut	Action
Windows + R	Open the Run dialog
Windows + E	Open File Explorer
Ctrl+Alt+Shift	Open the Guacamole side menu

Tips and Common Mistakes

Do

- **Do** include your pod prefix (PXX-) in every account name you create
- **Do** save files to the exact paths shown, **including your PodXX folder**
- **Do** use the correct extensions (.txt, .csv, .html)
- **Do** re-open a saved file to confirm the content landed
- **Do** put a date and your name in evidence files

Don't

- **Don't** save to C:\CyberLab\IA-Artifacts\ — it must be C:\CyberLab\PodXX\IA-Artifacts\
- **Don't** delete accounts when the lab says disable — they are different actions, and the checker looks for the disabled account
- **Don't** overwrite Device_Config_Record.csv in M4-L2 — append to it
- **Don't** modify objects outside your pod's OU or other students' objects
- **Don't** panic at red PowerShell text — read it, it usually names the problem

Common Issues

Problem	Solution
"Access denied" when saving	Save under C:\CyberLab\PodXX\, not a protected system folder
Verification says an account was not found	You probably left off the PXX- prefix — rename or recreate the account
Verification says a file was not found	Check for the PodXX folder in the path, and for a hidden .txt on your .csv
Can't find a user in ADUC	Edit → Find , search for your prefix (e.g. P03-)
PowerShell AD commands not recognized	Run <code>Import-Module ActiveDirectory</code> first
Task Scheduler won't save the change	Enter the service account password when prompted
File saved as .txt instead of .csv	In Save As, set Save as type to All Files (*.*)
New password rejected as too weak	The domain requires 12+ characters with complexity (M3-L2)

Lab Completion Checklist

Lab	Name	Status
M1-L1	Shared Reception Account	☐
M1-L2	Zombie Account	☐
M1-L3	Generic Accounts Present	☐
M2-L1	Scheduled Task Running as a Human Account	☐
M2-L2	Rogue Device Artifact	☐
M2-L3	Service Account Matrix Required	☐
M3-L1	Password Policy Report Missing	☐
M3-L2	Weak Password Policy	☐
M3-L3	User Must Change Password at Next Logon	☐
M4-L1	Default Credentials in the Hardening Standard	☐
M4-L2	SNMP Community String Set to "public"	☐
M4-L3	Script Contains a Hardcoded Password	☐

Evidence files you should end up with in C:\CyberLab\PodXX\: M1-L1.txt, M3-L1.txt, M3-L3.txt, M4-L1.txt, M4-L3.txt

Artifacts you should end up with in C:\CyberLab\PodXX\IA-Artifacts\:
Authorized_User_Inventory.csv, Authorized_Device_List.csv (updated),

Device_Config_Record.csv, Service_Account_Matrix.csv, PasswordPolicy_Report.html,
Hardening_Standard.txt (updated), Vault\Vault_Entries.txt

This guide was created for the Digital Resilience Community Clinic (DRCC) Cyber Range.
CMMC Level 1 — Identification & Authentication (IA) Module